

Sprawozdanie z laboratorium MPI

W ramach kursu:

Metody Programowania Równoległego

Stanisław Barycki

22 marca 2026

1 Pomiar komunikacji P2P

1.1 Strategia i pozyskanie danych

Celem eksperymentu było zmierzenie przepustowości i opóźnienia komunikacji punkt-punkt w klastrze vCluster przy użyciu `MPI_Send` i `MPI_Recv`. Pomiar przeprowadzono w dwóch konfiguracjach:

- **1 node** – dwa procesy na tej samej maszynie (komunikacja przez pamięć współdzieloną),
- **2 nodes** – po jednym procesie na dwóch różnych maszynach (komunikacja sieciowa).

Konfiguracje węzłów zapisano w plikach `onenode` i `twonode` (załącznik). Eksperymenty uruchomiono skryptem `run_pingpong.sh`:

```
mpiexec -machinefile ./onenode -np 2 ./ping_pong_c > ping_pong_result_1.csv
mpiexec -machinefile ./twonode -np 2 ./ping_pong_c > ping_pong_result_2.csv
```

Metoda pomiaru opiera się na schemacie **ping-pong**: proces 0 wysyła bufor do procesu 1, który natychmiast go odsyła. Operację powtórzono 1000 razy dla każdego rozmiaru komunikatu, a wynik uśredniono. Czas jednego pełnego round-tripu:

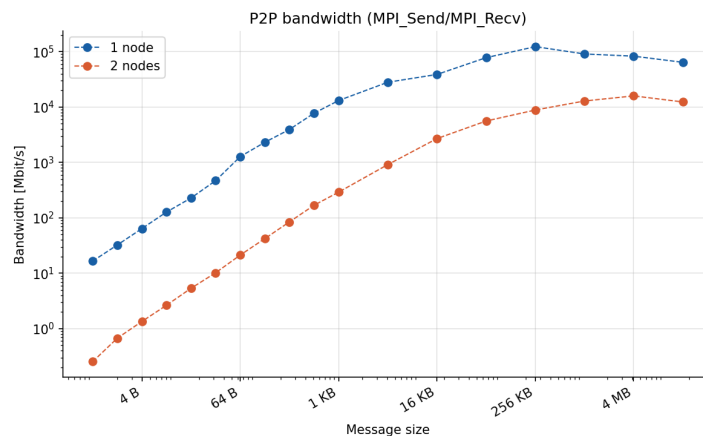
$$t_{rt} = \frac{T_{total}}{REPEATS}$$

Opóźnienie to połowa czasu round-tripu ($t_{rt}/2$), przepustowość zaś:

$$B = \frac{2 \cdot msg_size \cdot 8}{t_{rt} \cdot 10^6} \quad [\text{Mbit/s}]$$

Testowane rozmiary komunikatów: od 1 B do 16 MB (kolejne potęgi 2). Wykresy wygenerowano skryptem `plot_maker_pp.py`.

1.2 Wyniki – przepustowość

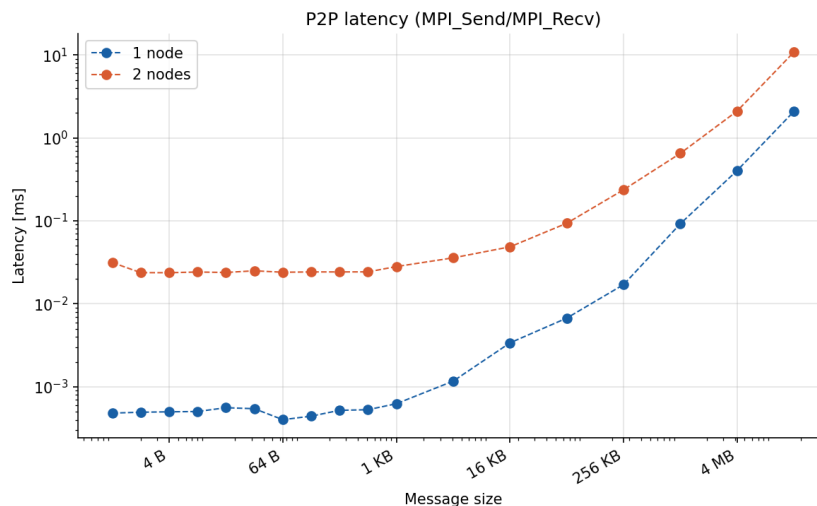


Rysunek 1: Przepustowość B w funkcji rozmiaru komunikatu dla obu konfiguracji. Obie osie w skali logarytmicznej.

Na rysunku 1 widoczna jest wyraźna różnica między konfiguracjami. Komunikacja w obrębie jednego węzła osiąga przepustowość rzędu 10^5 Mbit/s dla dużych komunikatów,

natomiast komunikacja między węzłami – około 10^4 Mbit/s, czyli rząd wielkości mniej. Dla małych komunikatów różnica sięga dwóch rzędów wielkości, co wynika z dominacji opóźnienia inicjalizacji przesyłu nad czasem faktycznego transferu danych. Obie krzywe wykazują wzrost przepustowości wraz z rozmiarem komunikatu – dla bardzo dużych wiadomości krzywe zaczynają się stabilizować, osiągając nasycenie przepustowości łącza.

1.3 Wyniki – opóźnienie



Rysunek 2: Opóźnienie w funkcji rozmiaru komunikatu. Obie osie w skali logarytmicznej.

Na rysunku 2 dla małych komunikatów opóźnienie jest praktycznie stałe – dominuje koszt stały nawiązania komunikacji, niezależny od rozmiaru danych. Dopiero od pewnego progu opóźnienie rośnie wraz z rozmiarem, co odpowiada fazie transferu danych. Próg ten jest wyraźnie wyższy dla komunikacji w obrębie jednego węzła.

Zmierzone opóźnienia dla komunikatu 1 B:

- **1 node:** 0.000482 ms ($\approx 0.5 \mu\text{s}$),
- **2 nodes:** 0.031458 ms ($\approx 31 \mu\text{s}$).

Komunikacja sieciowa jest zatem około 65 razy wolniejsza niż wewnątrz węzła dla krótkich komunikatów.

1.4 Wnioski

Wyniki potwierdzają że komunikacja wewnątrz jednego węzła jest znacznie wydajniejsza od komunikacji sieciowej. Różnica opóźnień (0.5 vs $31 \mu\text{s}$) wynika z tego, że komunikacja wewnętrzna odbywa się przez pamięć współdzieloną, eliminując narzut stosu sieciowego. Płynące z tego wnioski dla projektowania aplikacji równoległych: operacje wymagające częstej wymiany małych komunikatów powinny być lokalizowane w obrębie jednego węzła, natomiast komunikacja między węzłami powinna być ograniczona do niezbędnego minimum i – jeśli już konieczna – realizowana w postaci zbiorczych, dużych przesyłań, gdzie różnica przepustowości jest znacznie mniejsza.

2 Opis programu i algorytmu

Program przybliża wartość liczby π metodą Monte Carlo z użyciem biblioteki MPI. Algorytm polega na losowaniu N punktów $(x, y) \in [0, 1]^2$ i sprawdzaniu, czy należą do ćwiartki koła jednostkowego ($x^2 + y^2 \leq 1$). Przybliżenie wynika ze wzoru:

$$\pi \approx 4 \cdot \frac{\text{liczba punktów w kole}}{N}$$

Schemat komunikacji MPI

Każdy proces o numerze `rank` losuje niezależnie $\lfloor N/p \rfloor$ punktów (ostatni proces bierze również resztę z dzielenia). Każdy proces ma inny seed generatora liczb losowych, aby zróżnicować sekwencje losowanych punktów. Jedyna komunikacja to wywołanie `MPI_Reduce` sumującego lokalne liczniki do procesu 0 – złożoność komunikacyjna wynosi $O(\log p)$, a przesyłane są jedynie 8 bajty (`long long`). Narzut tej operacji jest pomijalny przy dłuższych obliczeniach, natomiast przy czasach rzędu mikrosekund może dominować nad rzeczywistą pracą.

Przyspieszenie względne i bezwzględne

We wszystkich eksperymentach stosowane jest **przyspieszenie względne**:

$$S(p) = \frac{T(1)}{T(p)}$$

gdzie $T(1)$ to czas programu MPI uruchomionego na jednym procesie. Przyspieszenie bezwzględne wymagałoby czasu implementacji czysto sekwencyjnej (bez MPI). W tym programie różnica jest pomijalna, ponieważ jedyną sekcją sekwencyjną jest `MPI_Reduce` trwające kilka μs . W ogólnym przypadku wybór ma znaczenie: przy istotnym narzucie inicjalizacji MPI lub kodzie wykonywanym tylko przez proces 0 przyspieszenie względne może wyglądać gorzej od bezwzględnego.

Prawa Amdahla i Gustafsona

Prawo Amdahla ogranicza maksymalne przyspieszenie przy stałym rozmiarze problemu (skalowanie silne):

$$S(p) \leq \frac{1}{f + \frac{1-f}{p}}$$

gdzie f to ułamek czasu spędzonego na operacjach niemożliwych do zrównoleglenia. Przy małych N stały narzut komunikacyjny stanowi istotny ułamek f , ograniczając $S(p)$ i powodując wypłaszczenie na wykresie.

Prawo Gustafsona dotyczy skalowania słabego – zakłada stałą ilość pracy na proces i przewiduje:

$$S(p) = p - f \cdot (p - 1)$$

Idealne zachowanie to $S(p) = 1$ (stały czas niezależnie od p). Spełnienie tego prawa wymaga odpowiednio dużego N/p , tak aby narzut komunikacji był pomijalny względem obliczeń.

3 Eksperymenty na vCluster

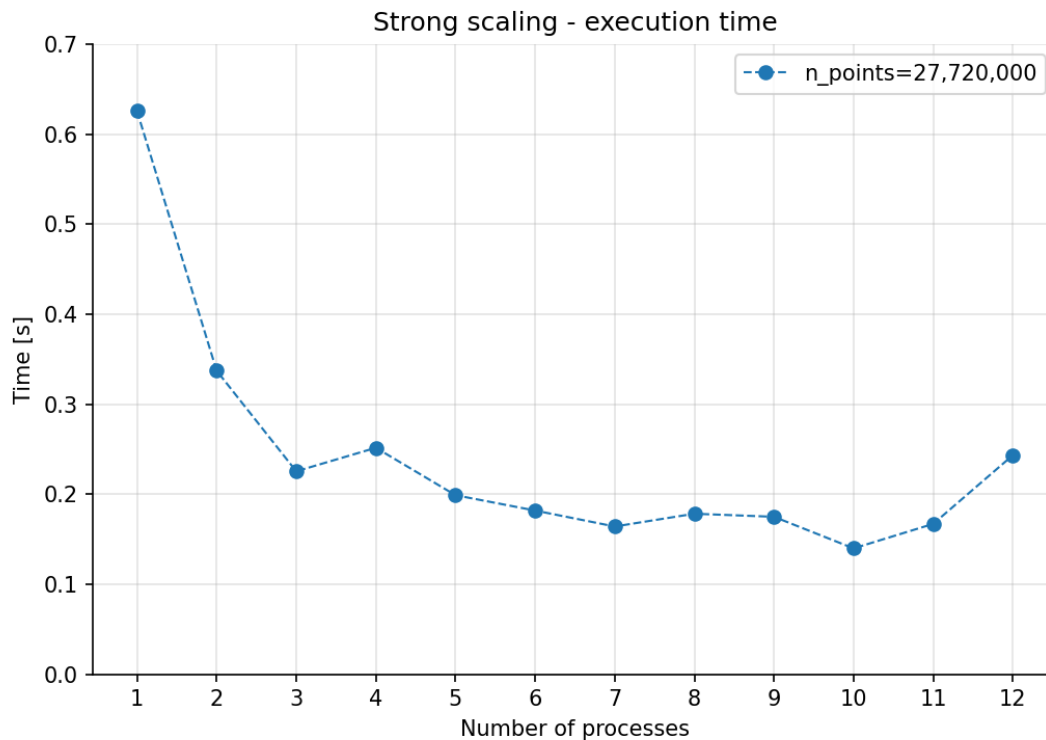
Środowisko vCluster zostało wykorzystane wyłącznie do weryfikacji poprawności działania programu. Właściwe pomiary wykonano na klastrze Ares (sekcja 4).

3.1 Pozyskanie danych

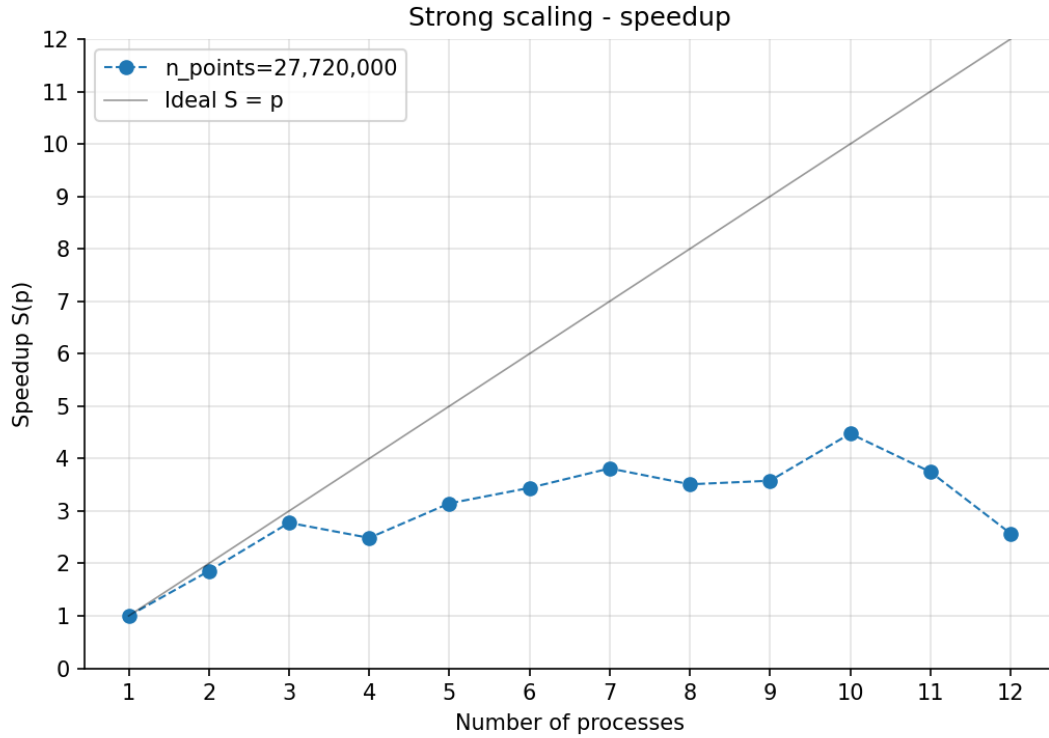
Eksperymenty uruchomiono skryptem `scaling_vcluster.sh` (załącznik) dla zakresu $p \in \{1, 2, \dots, 12\}$ procesów. Rozmiar problemu to $N = 27\,720\,000$ punktów dla skalowania silnego oraz $N/p = 2\,310\,000$ punktów na proces dla skalowania słabego. Wykonano jedno powtórzenie. Wykresy wygenerowano skryptem `plot_maker.py`.

3.2 Skalowanie silne – wyniki

Na rysunku 3 widoczny jest ogólny spadek czasu wraz ze wzrostem p , jednak z wyraźnymi nieregularnościami (np. wzrost czasu między $p = 3$ a $p = 4$ oraz przy $p = 12$). Na rysunku 4 przyspieszenie osiąga maksimum $S \approx 4.5$ przy $p = 10$, zamiast oczekiwanego $S = 10$. Czas $T(1) \approx 0.63$ s jest zbyt krótki – narzut komunikacji między węzłami dominuje, co zgodnie z prawem Amdahla powoduje obniżenie przyspieszenia. Wyniki mają jedynie charakter orientacyjny, ilustrując wpływ niedopasowania rozmiaru problemu do liczby procesów.



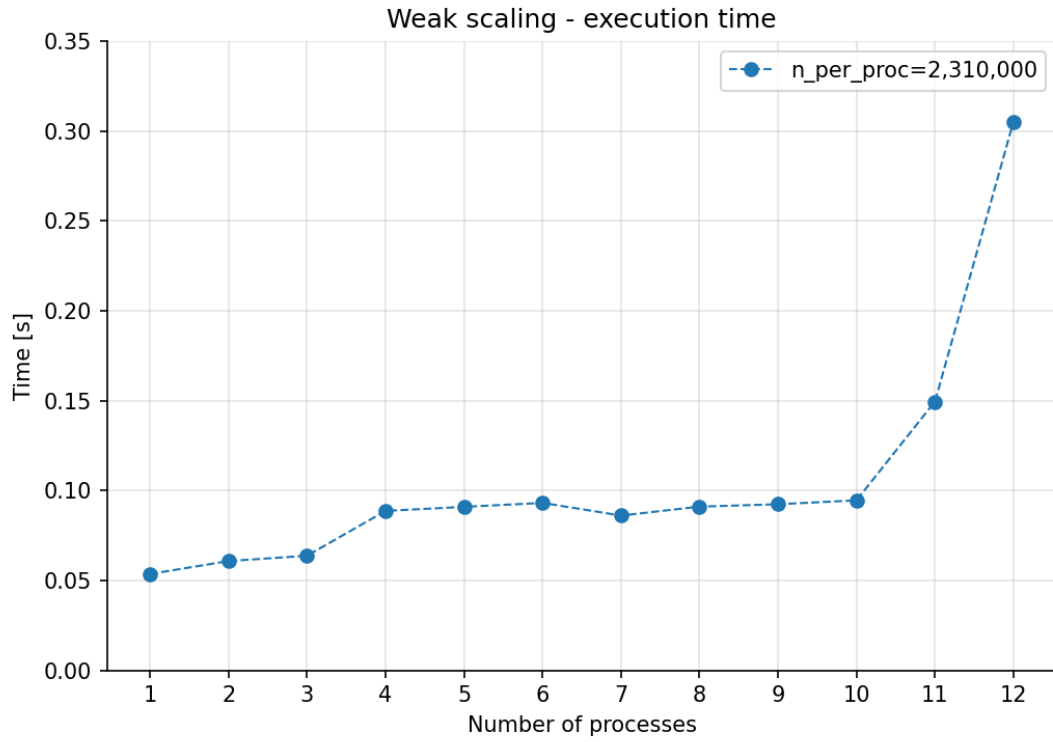
Rysunek 3: Skalowanie silne (vCluster) – czas wykonania $T(p)$ dla $N = 27\,720\,000$.



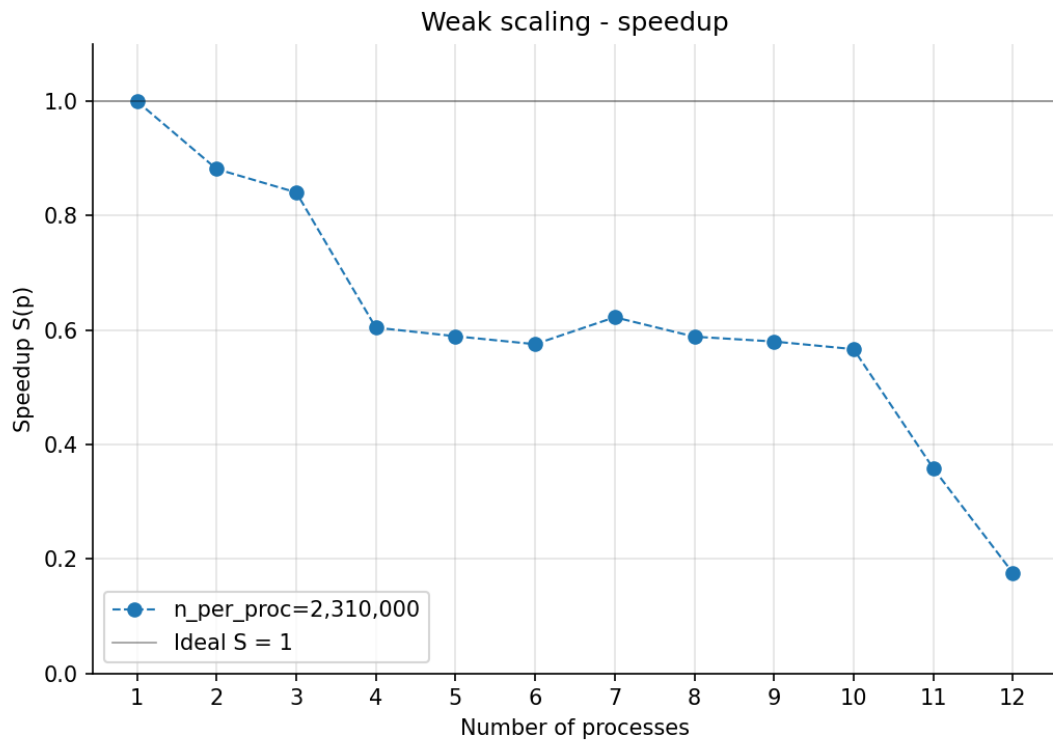
Rysunek 4: Skalowanie silne (vCluster) – przyspieszenie $S(p)$. Linia szara: idealne $S = p$.

3.3 Skalowanie słabe – wyniki

Czas rośnie wraz z p , szczególnie gwałtownie przy $p \in \{11, 12\}$, gdzie osiąga $T \approx 0.30$ s wobec $T(1) \approx 0.05$ s. Przyspieszenie spada do $S \approx 0.17$ przy $p = 12$, podczas gdy ideał to $S = 1$. Prawo Gustafsona nie jest spełnione – $N/p = 2\,310\,000$ jest zbyt małe, by narzut komunikacyjny był pomijalny. Wyniki potwierdzają poprawność działania programu, lecz nie nadają się do analizy wydajnościowej.



Rysunek 5: Skalowanie słabe (vCluster) – czas wykonania $T(p)$ dla $N/p = 2\,310\,000$.



Rysunek 6: Skalowanie słabe (vCluster) – przyspieszenie $S(p)$. Linia szara: idealne $S = 1$.

4 Eksperymenty na klastrze Ares

4.1 Pozyskanie danych

Pomiary przeprowadzono na klastrze Ares (ACK Cyfronet AGH), jeden węzeł obliczeniowy (`-nodes 1, -ntasks 12`), program `concurrency_montecarlo_2.c`, kompilacja `mpicc -O2`. Kolejowano 12 niezależnych powtórzeń przez `submit_all.sh` – każde jako osobny job SLURM, co eliminuje wpływ jednorazowych anomalii sprzętowych.

Napotkany błąd i jego konsekwencje

Kompilacja programu została początkowo umieszczona wewnątrz skryptu `run_ares.sh`, kolejowanego wielokrotnie równolegle. Spowodowało to *race condition* – joby jednocześnie nadpisywały plik wykonywalny `program_c`, co skutkowało błędami i brakiem wyników w połowie powtórzeń. Błąd naprawiono przenosząc kompilację do `submit_all.sh`, wykonywanego jednokrotnie przed kolejkowaniem. Jest to pouczający przykład problemu współbieżnego dostępu do zasobów w środowisku HPC. Pliki zawierające dane z dwóch nakładających się przebiegów przetworzono skryptem `cut_files.sh`, zachowując ostatnie 72 wiersze każdego pliku. Wyniki scalone poleceniem:

```
echo "scaling,n_points,procs,rep,time" > results/ares/raw_all.csv
cat results/ares/raw_rep*.csv >> results/ares/raw_all.csv
```

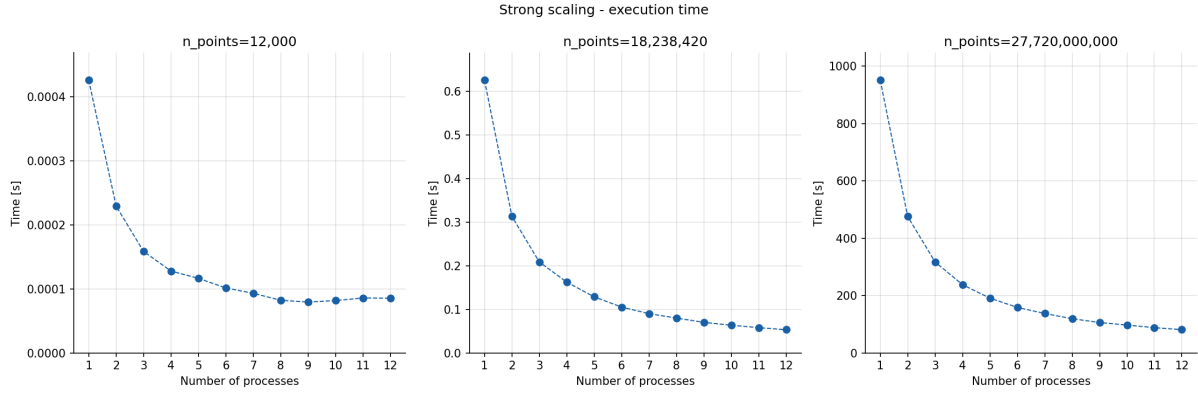
Wykresy wygenerowano skryptem `plot_maker_ares.py`.

4.2 Rozmiary problemów

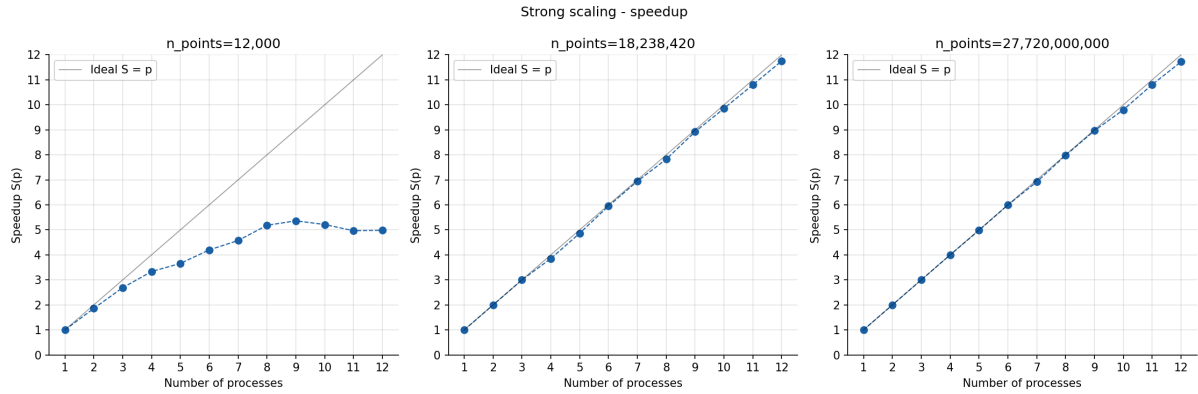
Tabela 1: Rozmiary problemów użyte w eksperymentach na Aresie.

Oznaczenie	N	$T(p = 1)$	$T(p = 12)$
Mały	12 000	≈ 0.43 ms	≈ 0.075 ms
Średni	18 238 420	≈ 0.63 s	≈ 0.052 s
Duży	27 720 000 000	≈ 952 s	≈ 79 s

4.3 Skalowanie silne – wyniki



Rysunek 7: Skalowanie silne (Ares) – czas wykonania $T(p)$. Osobna skala Y dla każdego rozmiaru.

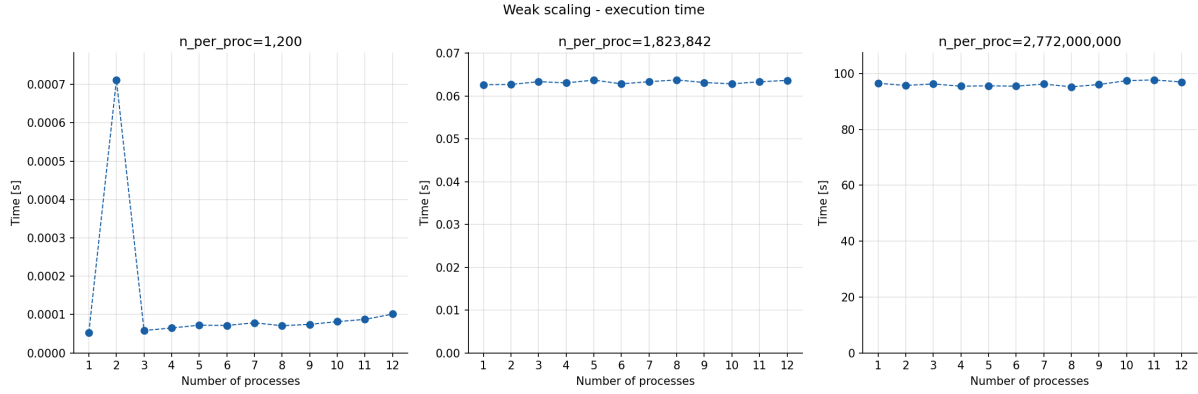


Rysunek 8: Skalowanie silne (Ares) – przyspieszenie $S(p)$. Linia szara: idealne $S = p$.

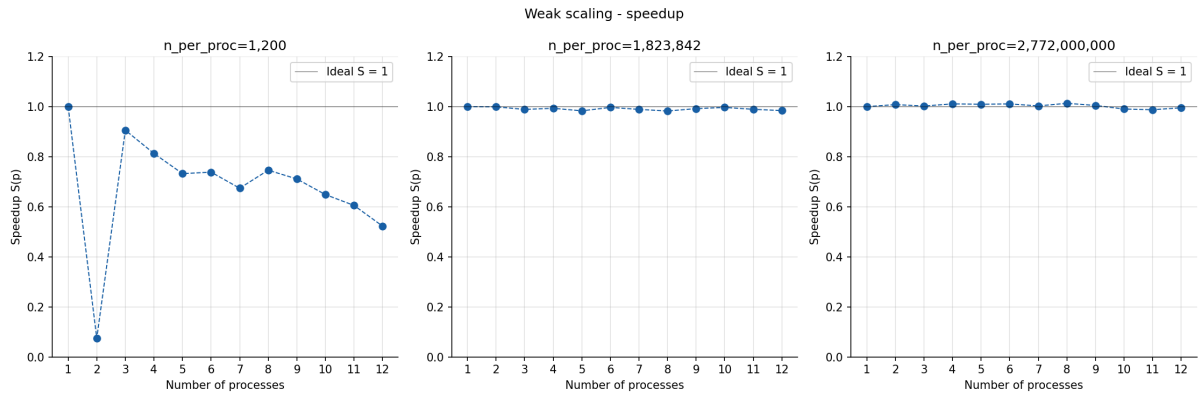
Na rysunku 7 czas spada monotonicznie dla wszystkich rozmiarów, zbliżając się do krzywej $T(1)/p$. Na rysunku 8:

- **Mały** ($N = 12\,000$): wykres przyspieszenia stabilizuje się na poziomie $S \approx 5$ od $p = 6$ – zgodnie z prawem Amdahla stały narzut MPI stanowi dominujący ułamek czasu przy $T(1) \approx 0.43$ ms.
- **Średni** ($N = 18\,238\,420$): przyspieszenie niemal idealne, $S \approx 12$ przy $p = 12$.
- **Duży** ($N = 27\,720\,000\,000$): zachowanie analogiczne do średniego, potwierdza klasyfikację algorytmu jako *embarrassingly parallel*.

4.4 Skalowanie słabe – wyniki



Rysunek 9: Skalowanie słabe (Ares) – czas wykonania $T(p)$.



Rysunek 10: Skalowanie słabe (Ares) – przyspieszenie $S(p)$. Linia szara: idealne $S = 1$.

- **Mały** ($N/p = 1\,200$): gwałtowny spadek do $S \approx 0.08$ przy $p = 2$, po czym częściowe odbudowanie. Obliczenia trwają $\sim 50\,\mu s$ – koszt inicjalizacji komunikacji dominuje. Prawo Gustafsona nie jest spełnione.
- **Średni** ($N/p = 1\,823\,842$): $S \approx 1$ przez cały zakres p , odchylenia rzędu 1%. Niemal idealne zachowanie zgodne z prawem Gustafsona.
- **Duży** ($N/p = 2\,772\,000\,000$): przyspieszenie miejscami przekracza $S = 1$. Możliwym wyjaśnieniem jest efekt cache – przy $p = 1$ jeden proces operuje na znacznie większym zbiorze danych, co może zwiększać liczbę cache miss. Możliwe jest również że są to wahania wynikające ze zmienności obciążenia węzła obliczeniowego.

4.5 Wnioski

Algorytm Monte Carlo jest *embarrassingly parallel* – przy odpowiednio dużym N uzyskuje się przyspieszenie bliskie idealnemu zarówno dla skalowania silnego jak i słabego. Kluczowy wniosek to dobór rozmiaru problemu: zbyt małe N powoduje dominację narzutów inicjalizacyjnych i komunikacyjnych, czyniąc zrównoleglenie bezcelowym. Prawo Amdahla wyjaśnia wypłaszczenie wykresu przyspieszenia dla małego problemu w skalowaniu silnym, prawo Gustafsona potwierdza się dla średniego i dużego w skalowaniu słabym.

Załączniki

Kod i skrypty dołączone są osobno. Do sprawozdania oprócz wymienionych tutaj explicit plików dołączam plik `.zip` całego folderu projektu wraz z wynikami poszczególnych pomiarów oraz wykresami:

- `ping_pong.c` – program pomiarowy P2P
- `run_pingpong.sh` – skrypt uruchamiający pomiary P2P
- `onenode`, `twonode` – pliki konfiguracji węzłów
- `plot_maker_pp.py` – wykresy P2P
- `results/pp/ping_pong_result_1.csv` – dane P2P, 1 node
- `results/pp/ping_pong_result_2.csv` – dane P2P, 2 nodes
- `concurrency_montecarlo_2.c` – kod programu MPI
- `scaling_vcluster.sh` – skrypt pomiarowy na vCluster
- `run_ares.sh` – jeden job SLURM = jedno powtórzenie
- `submit_all.sh` – kolejkuje powtórzenia na Aresie
- `cut_files.sh` – przetwarzanie plików po błędzie kompilacji
- `plot_maker.py` – wykresy vCluster
- `plot_maker_ares.py` – wykresy Ares
- `results/vcluster/raw_all.csv` – dane z vCluster
- `results/ares/raw_all.csv` – dane z Aresa